```
L3 = kron_sum([fdm_vertex(len(a), a[-1])[0] for a in ax3])
case("3차원 FDM 5×4×6", L3, tuple(ax3), "FDM")
Lb = L3.copy(); Lb[31, 31 + 5] *= 0.9; Lb[31, 31] += 0.1 * L3[31, 31 + 5]
case("3차원 FDM 버그", Lb, tuple(ax3), "FDM", "마디 31의 y 방향 결합 하나를 0.9배(행합은 맞춤)")
cx = [fvm_cell(4, 1.0)[2], fvm_cell(3, 1.0)[2], fvm_cell(5, 1.0)[2]]
Lv = kron_sum([fvm_cell(len(a), 1.0)[0] for a in cx])
case("3차원 FVM 4×3×5", Lv, tuple(cx), "FVM")
Lb = Lv.copy(); j = 13 + 12; Lb[13, j] = 0.0; Lb[13, 13] += Lv[13, j]
case("3차원 FVM 면 버그", Lb, tuple(cx), "FVM", "칸 13의 z 방향 면 하나를 빠뜨림(그 칸에서만, 행합은 맞춤)")
gx = [fem_p1_graded(5, 1.0)[2], fem_p1_graded(4, 1.0, 0.3)[2], np.array([0.0, 0.1, 0.4, 1.0])]
Lf3 = kron_sum([fem_p1(a)[0] for a in gx])
case("3차원 FEM P1(축별, 비균일) 5×4×4", Lf3, tuple(gx), "FEM P1")
Lb = Lf3.copy(); Lb[50] *= 1.2
case("3차원 FEM 질량 버그", Lb, tuple(gx), "FEM P1", "마디 50의 집중 질량을 1/1.2배로")

# ---------------------------------------------------------------- 비선형·변수 계수: -div(c(x, u) grad u)
rng = np.random.default_rng(3)
cfun = lambda xm, um: Fr(1, 20) * (1 + um * um)                    # c(u) = 0.05 (1 + u^2)
xs2 = (np.linspace(0, 1, 6), np.linspace(0, 1, 5)); V2, T2 = SC.claim_graph("FVM", xs2)
def fvm2_nonlinear(bug=False):
    """대상 코드(2차원 FVM, 비선형 확산): 면마다 c((u_i + u_j)/2) (u_j - u_i) * 전달률 / 칸 부피.  bug: 면 계수를 c(u_i)로(비대칭)"""
    Tf = {k: float(v) for k, v in T2.items()}; Vf = [float(v) for v in V2]
    def Fs(u):
        out = np.zeros(len(u))
        for (i, j), t in Tf.items():
            ci = 0.05 * (1 + u[i] ** 2) if bug else 0.05 * (1 + ((u[i] + u[j]) / 2) ** 2)
            cj = 0.05 * (1 + u[j] ** 2) if bug else ci
            out[i] += t * ci * (u[j] - u[i]) / Vf[i]; out[j] += t * cj * (u[i] - u[j]) / Vf[j]
        return out
    return Fs
states = [rng.uniform(0.2, 1.5, 30) for _ in range(3)]
from fractions import Fraction as Fr
for bug in (False, True):
    r = SC.check_flux(fvm2_nonlinear(bug), xs2, "FVM", cfun, states)
    fx = SC.check_flux(fvm2_nonlinear(False), xs2, "FVM", cfun, states) if bug else None
    print(format_report(SC.report(r, xs2, "2차원 FVM 비선형 확산", fx), title="[2차원 FVM, c(u) = 0.05(1 + u²)" + (", 면 계수를 c(u_i)로 쓴 버그]" if bug else "]")), flush=True); print()
cx1 = fem_p1_graded(11, 1.0)[2]; V1, T1 = SC.claim_graph("FEM P1", cx1)
def fem_varcoef(bug=False):
    """대상 코드(1차원 P1, 변수 계수 c(x) = 1 + x): 요소 가운데의 c.  bug: 요소 왼쪽 마디의 c를 씀"""
    def Fs(u):
        out = np.zeros(len(u))
        for (i, j), t in T1.items():
            c = 1 + (cx1[i] if bug else (cx1[i] + cx1[j]) / 2)
            out[i] += float(t) * c * (u[j] - u[i]) / float(V1[i]); out[j] += float(t) * c * (u[i] - u[j]) / float(V1[j])
        return out
    return Fs
for bug in (False, True):
    st1 = [np.random.default_rng(5).uniform(0, 1, 11)]
    r = SC.check_flux(fem_varcoef(bug), cx1, "FEM P1", lambda xm, um: 1 + xm[0], st1)
    fx = SC.check_flux(fem_varcoef(False), cx1, "FEM P1", lambda xm, um: 1 + xm[0], st1) if bug else None
    print(format_report(SC.report(r, cx1, "1차원 P1 변수 계수", fx), title="[1차원 FEM P1, c(x) = 1 + x" + (", 계수를 요소 왼쪽 마디에서 쓴 버그]" if bug else "]")), flush=True); print()
```

작동 확인 verify_space.py — A6: 검사 순서(공간 → 시간 → 정확도)

```
"""검사 순서(공간 -> 시간 -> 정확도): 같은 RK4(맞는 시간 스킴)를 맞는 공간 연산자와 틀린 공간 연산자 위에서 판정한다."""
import sys, numpy as np
sys.path.insert(0, "../numide")
from schemes_lib import rk4, u0
from operators import fdm_vertex
from semidisc import Sys
from verify import verify, library
from six_w import format_report
L, w, x = fdm_vertex(21, 1.0); R = lambda s: [s[0] * (1 - s[0]) - s[0] * s[1], 0.5 * s[0] * s[1] - 0.3 * s[1]]
case = sys.argv[1]
if case == "bug": L = L.copy(); L[12, 13] *= 1.3; L[12, 12] -= 0.3 * L[12, 13] / 1.3      # 마디 12의 오른쪽 결합 1.3배(행합은 0으로 맞춤)
S = Sys(dict(m=2, N=21, L=L, w=w, D=[0.02, 0.01], T=2.0, u0=u0, R=R))
rep = verify(S, step=rk4, N=40, claim=library()["RK4"], claim_name="RK4", eps=1e-5, name="RK4", space=dict(kind="FDM", x=x))
print(format_report(rep, title=f"[RK4(40걸음), 공간 연산자 {'버그' if case == 'bug' else '정상'}]"))
```

작동 확인 pinn_model.py — 17절: 시험용 PINN(대상 쪽 코드, 1–3차원; 검증기는 보지 않는다)

```
"""시험용 PINN(대상 쪽 코드, 검증기는 이 파일을 보지 않는다): u_t = D u_xx + u - u^2, 영 유량, u(x, 0) = 0.5 + 0.3 cos(pi x).
은닉층 하나짜리 tanh 신경망, 입력 (cos(pi x), 2t/T - 1).  초기 조건과 영 유량 경계는 꼴로 강제한다:
    u(x, t) = u0(x) + t * N(cos(pi x), 2t/T - 1)       (d/dx cos(pi x) = 0 이 x = 0, 1에서 성립)
학습은 콜로케이션 점의 잔차 u_t - D u_xx - (u - u^2)에 대한 레벤버그–마쿼트(scipy.optimize.least_squares).
검증기에 넘기는 것은 함수 u(x, t)뿐이다(가중치·손실·구조는 넘기지 않는다)."""
import numpy as np
from scipy.optimize import least_squares

D, T = 0.05, 0.5
u0 = lambda x: 0.5 + 0.3 * np.cos(np.pi * x)
u0xx = lambda x: -0.3 * np.pi ** 2 * np.cos(np.pi * x)


class PINN:
    def __init__(s, H=20, seed=0):
        r = np.random.default_rng(seed); s.H = H
        s.th = np.concatenate([r.normal(0, 1, 2 * H), r.normal(0, 0.5, H), r.normal(0, 0.1, H), [0.0]])

    def parts(s, th):
        H = s.H; return th[:H], th[H:2 * H], th[2 * H:3 * H], th[3 * H:4 * H], th[4 * H]   # a(cos), b(time), bias, w, b2

    def fields(s, th, x, t):
        a, b, c, w, b2 = s.parts(th); C = np.cos(np.pi * x); tt = 2 * t / T - 1
        z = np.outer(C, a) + np.outer(tt, b) + c; th_ = np.tanh(z); sd = 1 - th_ ** 2
        N = th_ @ w + b2
        Nt = (sd * b) @ w * (2 / T)
        Cx, Cxx = -np.pi * np.sin(np.pi * x), -np.pi ** 2 * C
        Nc = (sd * a) @ w; Ncc = (-2 * th_ * sd * a ** 2) @ w
        Nxx = Ncc * Cx ** 2 + Nc * Cxx
        u = u0(x) + t * N; ut = N + t * Nt; uxx = u0xx(x) + t * Nxx
        return u, ut, uxx

    def residual(s, th, x, t):
        u, ut, uxx = s.fields(th, x, t); return ut - D * uxx - (u - u * u)

    def train(s, xc, tc, nfev):
        r = least_squares(lambda th: s.residual(th, xc, tc), s.th, method="lm", max_nfev=nfev, xtol=1e-15, ftol=1e-15, gtol=1e-15)
        s.th = r.x; return float(np.sqrt(np.mean(r.fun ** 2)))

    def __call__(s, x, t):
        x = np.asarray(x, float); return s.fields(s.th, x, np.full(x.shape, float(t)))[0]


def colloc(nx=24, nt=16, extra=None, seed=1):
    r = np.random.default_rng(seed); X, Tt = np.meshgrid(np.linspace(0, 1, nx), np.linspace(0, T, nt)); xc, tc = X.ravel(), Tt.ravel()
    if extra is not None:                                   # (t0, t1, x0, x1, n): 국소화가 짚은 자리에 콜로케이션 점을 더한다
        t0, t1, x0, x1, n = extra; xc = np.append(xc, r.uniform(x0, x1, n)); tc = np.append(tc, r.uniform(t0, t1, n))
    return xc, tc


class PINNND:
    """d차원 판: u_t = D Δu + u - u^2, [0,1]^d, 모든 면에서 영 유량, u(x, 0) = 0.5 + 0.3 prod_k cos(pi x_k).
    u(x, t) = u0(x) + t * N(cos(pi x_1), ..., cos(pi x_d), 2t/T - 1)  (초기 조건과 영 유량을 꼴로 강제)"""
    def __init__(s, d, H=20, seed=0):
        r = np.random.default_rng(seed); s.d, s.H = d, H
        s.th = np.concatenate([r.normal(0, 1, (d + 1) * H), r.normal(0, 0.1, H), r.normal(0, 0.1, H), [0.0]])

    def parts(s, th):
        d, H = s.d, s.H; W = th[:(d + 1) * H].reshape(d + 1, H); o = (d + 1) * H
        return W[:d], W[d], th[o:o + H], th[o + H:o + 2 * H], th[o + 2 * H]

    @staticmethod
    def u0(X): return 0.5 + 0.3 * np.prod(np.cos(np.pi * X), axis=0)

    def fields(s, th, X, t):
        A, b, c, w, b2 = s.parts(th); C = np.cos(np.pi * X); tt = 2 * t / T - 1          # X: (d, P)
```

```
        z = C.T @ A + np.outer(tt, b) + c; th_ = np.tanh(z); sd = 1 - th_ ** 2
        N = th_ @ w + b2; Nt = (sd * b) @ w * (2 / T)
        lapN = np.zeros(X.shape[1]); lap0 = np.zeros(X.shape[1]); u0v = s.u0(X)
        for k in range(s.d):
            Cx, Cxx = -np.pi * np.sin(np.pi * X[k]), -np.pi ** 2 * C[k]
            lapN += ((-2 * th_ * sd * A[k] ** 2) @ w) * Cx ** 2 + ((sd * A[k]) @ w) * Cxx
            lap0 += -np.pi ** 2 * u0v + np.pi ** 2 * 0.5      # d²/dx_k² of 0.3 prod cos = -pi² (u0 - 0.5)
        return u0v + t * N, N + t * Nt, lap0 + t * lapN

    def residual(s, th, X, t):
        u, ut, lap = s.fields(th, X, t); return ut - D * lap - (u - u * u)

    def train(s, Xc, tc, nfev):
        r = least_squares(lambda th: s.residual(th, Xc, tc), s.th, method="lm", max_nfev=nfev, xtol=1e-15, ftol=1e-15, gtol=1e-15)
        s.th = r.x; return float(np.sqrt(np.mean(r.fun ** 2)))

    def __call__(s, X, t):
        X = np.asarray(X, float); return s.fields(s.th, X, np.full(X.shape[1], float(t)))[0]


def colloc_nd(d, nx=None, nt=12, extra=None, seed=1):
    nx = nx or {1: 24, 2: 10, 3: 7}.get(d, 5); r = np.random.default_rng(seed); g = np.meshgrid(*([np.linspace(0, 1, nx)] * d + [np.linspace(0, T, nt)]), indexing="ij")
    X = np.stack([a.ravel() for a in g[:d]]); tc = g[d].ravel()
    if extra is not None:                    # (t0, t1, [(x0, x1)]*d, n)
        t0, t1, box, n = extra; Xe = np.stack([r.uniform(a, b, n) for a, b in box]); X = np.hstack([X, Xe]); tc = np.append(tc, r.uniform(t0, t1, n))
    return X, tc
```

작동 확인 pinn_nd_test.py — 17절: PINN 출력과 방정식의 어긋남 판정(차원 무관)

```
"""PINN 출력과 방정식의 어긋남을 차원과 상관없이: d = 1, 2, 3에서 같은 PDE의 PINN(검은 상자)을 상수 후보로 바꿔 판정한다.
    u_t = 0.05 Δu + u - u², [0,1]^d, 영 유량, u(x, 0) = 0.5 + 0.3 prod cos(pi x_k), T = 0.5"""
import sys, time, numpy as np
sys.path.insert(0, "../numide")
from pde_judge_nd import JudgeND
from orders_nd import mol_nd, kron_operator
from pinn_cert import certify_with_prescription_nd
from pinn_model import PINNND, colloc_nd, D, T
from six_w import format_report
R = lambda s: [s[0] - s[0] ** 2]
def f0(X):
    p = 1.0
    for x in X: p = p * np.cos(np.pi * x)
    return 0.5 + 0.3 * p
def u0box(a, xl, xh):                        # cos(pi x) is decreasing on [0, 1]: interval product of [cos(pi xh), cos(pi xl)]
    lo = np.ones_like(xl[0]); hi = np.ones_like(xl[0])
    for l, h_ in zip(xl, xh):
        cl, ch = np.cos(np.pi * np.clip(h_, 0, 1)), np.cos(np.pi * np.clip(l, 0, 1))
        c = np.stack([lo * cl, lo * ch, hi * cl, hi * ch]); lo, hi = c.min(0), c.max(0)
    return 0.5 + 0.3 * lo - 1e-14, 0.5 + 0.3 * hi + 1e-14
def u0grad(a, xl, xh):                 # d/dx_i f0 = -0.3 pi sin(pi x_i) prod_{j != i} cos(pi x_j): interval products
    d = len(xl); cosI = []; sinI = []
    for l, h_ in zip(xl, xh):
        l = np.clip(l, 0, 1); h_ = np.clip(h_, 0, 1)
        cosI.append((np.cos(np.pi * h_) - 1e-15, np.cos(np.pi * l) + 1e-15))
        sl, sh = np.sin(np.pi * l), np.sin(np.pi * h_); smax = np.where((l <= 0.5) & (h_ >= 0.5), 1.0, np.maximum(sl, sh))
        sinI.append((np.minimum(sl, sh) - 1e-15, smax + 1e-15))
    out = []
    for i in range(d):
        lo, hi = sinI[i]
        for j in range(d):
            if j == i: continue
            c = np.stack([lo * cosI[j][0], lo * cosI[j][1], hi * cosI[j][0], hi * cosI[j][1]]); lo, hi = c.min(0), c.max(0)
        k = -0.3 * np.pi; out.append((k * hi - 1e-14, k * lo + 1e-14))
    return out

d = int(sys.argv[1]); nfevs = [int(a) for a in sys.argv[2].split(",")]; eps = float(sys.argv[3])
cfg = {1: (41, 50, 5, 81, 400), 2: (17, 20, 4, 33, 80), 3: (9, 10, 4, 17, 40)}[d]; nx, n, K, nxr, ntr = cfg
spec = dict(m=1, D=[D], R=R, T=T, box=([0.0], [1.0])); op = lambda xs_: kron_operator(xs_)
t0 = time.time(); rxs = [np.linspace(0, 1, nxr)] * d
```

```
rp = mol_nd(dict(m=1, D=[D], R=R, T=T, u0=[f0]), rxs, ntr)
rres = JudgeND(dict(spec, dim=d)).run(rp, u0box, op=op, u0grad=u0grad)
print(f"[d = {d}] 기준해: 선 방법(RK4 에르미트) {nxr}^{d}마디, {ntr}조각, 경로 B 한도 {rres['bound']:.2e} ({time.time() - t0:.0f}초)", flush=True)
xs = [np.linspace(0, 1, nx)] * d
for nf in nfevs:
    p = PINNND(d); Xc, tc = colloc_nd(d); t1 = time.time(); rms = p.train(Xc, tc, nf); tt = time.time() - t1
    def retrain(region, p=p):
        """대상 쪽 처방: 짚은 구역에 콜로케이션 점 300개를 더해 이어서 학습(가중치를 이어받음)"""
        t0_, t1_, box = region; X2, t2 = colloc_nd(d, extra=(t0_, t1_, box, 300)); p.train(X2, t2, 300 if d < 3 else 800); return p
    t1 = time.time()
    rep, out = certify_with_prescription_nd(p, spec, u0box, u0grad, xs, n, K, eps, (rp, rres["series"]), op=op, name=f"PINN(d = {d}, 학습 {nf}회)", retrain=retrain, nmax=2 * nx)
    print(format_report(rep, title=f"[d = {d}, PINN 학습 {nf}회, 잔차 rms {rms:.1e}, 학습 {tt:.0f}초, 판정 {time.time() - t1:.0f}초]"), flush=True); print(flush=True)
```

작동 확인 gaps_test.py — 19절: 주장 없이 가려내기, 차수 조건, 조건부 버그 탐색

```
"""19절: 빈틈 셋(주장이 없을 때, 공식 자체가 틀렸을 때, 특정 조건에서만 드러날 때)을 메우는 검사를 시험한다."""
import sys, numpy as np
sys.path.insert(0, "../numide")
from fractions import Fraction as Fr
from hypothesis import strategies as st
from claim_tools import identify_space, rk_order_check, lmm_order_check, stencil_check, fornberg, tree_str, fuzz_find
from operators import fdm_vertex, fvm_cell, fem_p1, fem_p1_graded
import space_check as SC
from verify import check_claim, library
from schemes_lib import S, rk4

print("== 빈틈 1: 주장 없이 공간 이산을 목록에서 가려낸다")
def show(name, L, x):
    kept, ref = identify_space(L, x)
    why = "; ".join(f"{k}: 마디 {v['i']}, 이웃 {v['j']}" if v.get("i") is not None else f"{k}: {v.get('note', '')[:40]}" for k, v in ref.items())
    print(f"  {name}: 남은 이산 {kept if kept else '없음(반례: 목록의 어느 이산과도 맞지 않는다)'}  | 배제: {why}")
x = np.linspace(0, 1, 21); show("FDM(꼭짓점) 21마디", fdm_vertex(21, 1.0)[0], x)
show("FEM P1 균일 21마디", fem_p1(x)[0], x)
L, w, xc = fvm_cell(20, 1.0); show("FVM 20칸", L, xc)
Lb = L.copy(); Lb[7] /= 2; show("FVM 칸 너비 버그(주장 없음)", Lb, xc)
L, w, xg = fem_p1_graded(21, 1.0); show("FEM P1 비균일", L, xg)

print("\n== 빈틈 2: 공식 자체를 차수 조건과 대조한다(정확한 유리수)")
half = Fr(1, 2)
RK4 = ([[0, 0, 0, 0], [half, 0, 0, 0], [0, half, 0, 0], [0, 0, 1, 0]], [Fr(1, 6), Fr(1, 3), Fr(1, 3), Fr(1, 6)])
R38 = ([[0, 0, 0, 0], [Fr(1, 3), 0, 0, 0], [Fr(-1, 3), 1, 0, 0], [1, -1, 1, 0]], [Fr(1, 8), Fr(3, 8), Fr(3, 8), Fr(1, 8)])
TYPO = ([[0, 0, 0, 0], [half, 0, 0, 0], [0, half, 0, 0], [0, half, half, 0]], [Fr(1, 6), Fr(1, 3), Fr(1, 3), Fr(1, 6)])
for name, (A, b), p in (("고전 RK4", RK4, 4), ("3/8 규칙", R38, 4), ("'RK4'라 주장한 표(a43 = a42 = 1/2 오타)", TYPO, 4)):
    ok, bad = rk_order_check(A, b, p)
    print(f"  {name}: 차수 {p} 조건 {'모두 성립(증명)' if ok else '위반(반례)'}" + ("" if ok else ": " + ", ".join(f"나무 {tree_str(t)}(차수 {n}): bᵀΦ = {l}, 1/γ = {r}" for t, n, l, r in bad[:3])))
ok5, bad5 = rk_order_check(*RK4, 5); print(f"  고전 RK4에 '5차'를 주장하면: 차수 5 나무 {len(bad5)}개에서 위반(반례), 예: 나무 {tree_str(bad5[0][0])}: bᵀΦ = {bad5[0][2]}, 1/γ = {bad5[0][3]}")
for name, al, be, p in (("AB3", [0, 0, -1, 1], [Fr(5, 12), Fr(-16, 12), Fr(23, 12), 0], 3), ("'AB3'(23을 22로 쓴 오타)", [0, 0, -1, 1], [Fr(5, 12), Fr(-16, 12), Fr(22, 12), 0], 3),
                       ("BDF2", [Fr(1, 3), Fr(-4, 3), 1], [0, 0, Fr(2, 3)], 2)):
    ok, bad = lmm_order_check(al, be, p)
    print(f"  {name}: C_0..C_{p} {'= 0(증명)' if ok else '위반(반례)'}" + ("" if ok else ": " + ", ".join(f"C_{q} = {c}" for q, c in bad)))
w4 = fornberg([-2, -1, 0, 1, 2], 2); print(f"  포른베르그(정확한 유리수)가 만든 5점 2계 가중치: {[str(v) for v in w4]}")
for name, wts in (("4차 5점 u_xx", w4), ("'4차'라 주장한 5점(-30을 -31로)", [Fr(-1, 12), Fr(16, 12), Fr(-31, 12), Fr(16, 12), Fr(-1, 12)]), ("3점 u_xx에 '4차'를 주장", [0, 1, -2, 1, 0])):
    ok, bad = stencil_check([-2, -1, 0, 1, 2], wts, 2, 4 if "4차" in name else 2)
    print(f"  {name}: 모멘트 조건 {'성립(증명)' if ok else '위반(반례)'}" + ("" if ok else ": " + ", ".join(f"m_{r} = {m}" for r, m in bad)))

print("\n== 빈틈 3: 특정 조건에서만 드러나는 버그를 입력 탐색과 축소로 찾는다(오라클은 정확한 검사)")
def fdm_latent(N):
    """대상 코드: N > 40이면 가운데 마디의 결합을 1.001배로 만드는 숨은 버그"""
    L, w, x = fdm_vertex(N, 1.0)
```

```
    if N > 40: L = L.copy(); L[N // 2, N // 2 + 1] *= 1.001
    return L, x
N = fuzz_find(st.integers(3, 200), lambda N: not SC.check(*fdm_latent(N), "FDM")["ok"])
r = SC.check(*fdm_latent(N), "FDM"); v = max(r["viol"], key=lambda v: v["excess"])
print(f"  FDM(크기에 따라 숨은 버그): 최소 반례 N = {N}, 마디 {v['i']}, 이웃 {v['j']}, 규칙 {v['rule']}"
)

def fem_latent(a):
    """대상 코드: 집중 질량을 평균 간격으로 계산(균일 격자에서는 맞고 비균일에서만 틀림)"""
    L, w, x = fem_p1_graded(11, 1.0, a); hbar = (x[-1] - x[0]) / 10; wb = np.full(11, hbar); wb[0] = wb[
-1] = hbar / 2
    K = np.zeros((11, 11))
    for e in range(10): K[e:e + 2, e:e + 2] += np.array([[1, -1], [-1, 1]]) / (x[e + 1] - x[e])
    return -K / wb[:, None], x
a = fuzz_find(st.floats(0.0, 0.6), lambda a: not SC.check(*fem_latent(a), "FEM P1")["ok"])
r0 = SC.check(*fem_latent(0.0), "FEM P1")
lo, hi = 0.0, a                                      # 축소된 반례에서 경계까지 이분법(반례가 드러나는
가장 작은 비틀림)
for _ in range(40):
    mid = 0.5 * (lo + hi)
    if SC.check(*fem_latent(mid), "FEM P1")["ok"]: lo = mid
    else: hi = mid
print(f"  FEM P1(비균일 격자에서만 틀림): 균일(a = 0)에서는 통과 {r0['ok']}; 탐색이 줄인 반례 a = {a!r};
 이분법으로 찾은 경계: a > {hi:.3e}이면 반례")

def rk4_latent_run(Nst):
    """대상 코드: 걸음 수가 7의 배수일 때만 마지막 걸음에서 k3 대신 k2를 쓰는 숨은 버그"""
    Y = [S.u0.copy()]; h = S.T / Nst; F = S.F
    for k in range(Nst):
        y = Y[-1]; k1 = F(y); k2 = F(y + h / 2 * k1); k3 = F(y + h / 2 * k2)
        if Nst % 7 == 0 and k == Nst - 1: k3 = k2
        k4 = F(y + h * k3); Y.append(y + h / 6 * (k1 + 2 * k2 + 2 * k3 + k4))
    return np.linspace(0, S.T, Nst + 1), np.array(Y)
cl = library()["RK4"]
Nst = fuzz_find(st.integers(20, 300), lambda n: check_claim(S, *rk4_latent_run(n), cl)[0] == "refuted")
Nmin = next(n for n in range(20, Nst + 1) if check_claim(S, *rk4_latent_run(n), cl)[0] == "refuted")   #
 그 아래를 모두 확인
st_, info = check_claim(S, *rk4_latent_run(Nmin), cl)
print(f"  RK4(걸음 수가 7의 배수일 때만 숨은 버그): 탐색이 찾은 반례 걸음 수 {Nst}; 그 아래를 모두 확인
한 최소 반례 {Nmin}, 반례 걸음 {info[0]}, 초과 {info[1]:.2e}")
clean = [n for n in range(29, 35) if check_claim(S, *rk4_latent_run(n), cl)[0] == "refuted"]
print(f"  7의 배수가 아닌 걸음 수 29–34로 돌리면 반례 없음: {not clean} (한두 번의 시험으로는 놓친다)")
```

작동 확인 higher_order_test.py — 4.6절: 2계 이상(쌍조화, 4계 스텐실, 칸–힐리어드)의 공간 스킴 검사

```
"""4.6절: 2계 이상의 공간 스킴 — 쌍조화(4계)의 1·2·3차원 FDM·FVM·FEM, 4계 스텐실 가려내기, 비선형 4계(칸
–힐리어드)."""
import sys, warnings, numpy as np
warnings.filterwarnings("ignore")
sys.path.insert(0, "../numide")
from fractions import Fraction as Fr
import space_check as SC
from space_check import EV
from claim_tools import identify_stencil, fornberg, stencil_check
from operators import fdm_vertex, fvm_cell, fem_p1, fem_p1_graded
from six_w import format_report


def show(name, r, x, fix=None, note="", recheck=None):
    if fix is None and recheck is not None and not r["ok"]: fix = recheck(r)
    print(format_report(SC.report(r, x, name, fix), title=f"[{name}]" + (f"  — {note}" if note else ""))
, flush=True); print()


def kron_sum(Ls):
    d = len(Ls); n = [A.shape[0] for A in Ls]; out = 0
    for k in range(d):
        M = np.eye(1)
        for l in reversed(range(d)): M = np.kron(M, Ls[l] if l == k else np.eye(n[l]))
        out = out + M
    return out


# 1차원 쌍조화 u_xxxx: 대상 코드는 2계 연산자를 두 번 곱한다(경계는 영 유량 유령점을 두 번 쓴 꼴)
L, w, x = fdm_vertex(21, 1.0); B = L @ L
show("1차원 FDM 쌍조화(L²)", SC.check_power(B, x, "FDM", 2), x)
Bb = B.copy(); Bb[10, 10] *= 1 - 1 / 600
```

```
show("1차원 FDM 쌍조화 버그", SC.check_power(Bb, x, "FDM", 2), x, SC.check_power(SC.patched(Bb, SC.check_power(Bb, x, "FDM", 2)["viol"]), x, "FDM", 2), "마디 10의 대각 6/h⁴을 5.99/h⁴로")
Lg, wg, xg = fem_p1_graded(21, 1.0); Bg = Lg @ Lg
show("1차원 FEM P1 쌍조화(비균일)", SC.check_power(Bg, xg, "FEM P1", 2), xg)
Bgb = Lg @ (Lg * np.where(np.arange(21) == 7, 1.05, 1.0)[None, :])
show("1차원 FEM P1 쌍조화 버그", SC.check_power(Bgb, xg, "FEM P1", 2), xg, None, "안쪽 연산자의 열 7(마디 7의 값)을 1.05배로 읽음", lambda r: SC.check_power(SC.patched(Bgb, r["viol"]), xg, "FEM P1", 2))
# 2·3차원 쌍조화 Δ²: 텐서 격자
ax2 = (fvm_cell(6, 1.0)[2], fvm_cell(5, 1.0)[2]); L2 = kron_sum([fvm_cell(6, 1.0)[0], fvm_cell(5, 1.0)[0]])
show("2차원 FVM 쌍조화 6×5", SC.check_power(L2 @ L2, ax2, "FVM", 2), ax2)
L2b = L2.copy(); L2b[14, 15] *= 1.1; L2b[14, 14] -= 0.1 * L2[14, 15]
show("2차원 FVM 쌍조화 버그", SC.check_power(L2 @ L2b, ax2, "FVM", 2), ax2, None, "안쪽 라플라시안의 칸 14 → 15 면을 1.1배로", lambda r: SC.check_power(SC.patched(L2 @ L2b, r["viol"]), ax2, "FVM", 2))
ax3 = tuple(np.linspace(0, 1, n) for n in (5, 4, 4)); L3 = kron_sum([fdm_vertex(len(a), 1.0)[0] for a in ax3])
show("3차원 FDM 쌍조화 5×4×4", SC.check_power(L3 @ L3, ax3, "FDM", 2), ax3)
B3b = L3 @ L3; B3b[33, 33 + 20] += 2.0
show("3차원 FDM 쌍조화 버그", SC.check_power(B3b, ax3, "FDM", 2), ax3, None, "마디 33의 z 방향 이웃 항에 2를 더함", lambda r: SC.check_power(SC.patched(B3b, r["viol"]), ax3, "FDM", 2))
# 4계 스텐실을 포른베르그 목록으로 가려내기(균일 내부 행)
xu = np.linspace(0, 1, 21); h = xu[1] - xu[0]
for npts, bug in ((5, None), (7, None), (7, 10)):
    w = [float(v) for v in fornberg(list(range(-(npts // 2), npts // 2 + 1)), 4)]; M = np.zeros((21, 21)); r = npts // 2
    for i in range(r, 21 - r): M[i, i - r:i + r + 1] = np.array(w) / h ** 4
    if bug: M[bug, bug] *= 1.0001
    found, un = identify_stencil(M, xu, deriv=4, widths=(5, 7, 9))
    print(f"[4계 스텐실 {npts}점{', 행 10에 1e-4 오타' if bug else ''}] 가려낸 스텐실 {sorted(set(found.values()))}, 일치한 내부 행 {len(found)}개" + (f", 맞는 것이 없는 행 {un}(반례)" if un else ""))
ok, bad = stencil_check([-2, -1, 0, 1, 2], [1, -4, 6, -4, 1], 4, 2); print(f"[4계 5점 (1, -4, 6, -4, 1)의 2차 모멘트 조건] {'성립(증명)' if ok else bad}")
ok, bad = stencil_check([-2, -1, 0, 1, 2], [1, -4, 6, -4, 1], 4, 4); print(f"[같은 스텐실에 '4차'를 주장] {'성립' if ok else '위반(반례): ' + ', '.join(f'm_{r} = {m}' for r, m in bad)}"); print()
# 비선형 4계: 칸–힐리어드  u_t = Δ(u³ - u - κ Δu), 공간 항 F(u) = L(u³ - u - κ L u)
kap = Fr(1, 1000); xc = np.linspace(0, 1, 33)
def ch_code(bug=False):
    """대상 코드(1차원 FDM 칸–힐리어드)"""
    L1 = fdm_vertex(33, 1.0)[0]; Lo = L1.copy()
    if bug: Lo[0, 1] = Lo[0, 1] / 2                          # 바깥 연산자에서만 유령점의 2배를 빠뜨림
    return lambda u: Lo @ (u ** 3 - u - float(kap) * (L1 @ u))
formula = lambda Lx, u: Lx(u ** 3 - u - kap * Lx(u))
rng = np.random.default_rng(7); states = [0.6 * np.cos(np.pi * xc * k) + 0.05 * rng.standard_normal(33) for k in (1, 2, 3)]
r = SC.check_formula(ch_code(), xc, "FDM", formula, states, "칸–힐리어드 L(u³ - u - κLu)"); show("1차원 칸–힐리어드(비선형 4계)", r, xc)
r = SC.check_formula(ch_code(True), xc, "FDM", formula, states, "칸–힐리어드 L(u³ - u - κLu)")
fx = SC.check_formula(ch_code(False), xc, "FDM", formula, states, "칸–힐리어드 L(u³ - u - κLu)")
show("1차원 칸–힐리어드 버그", r, xc, fx, "바깥 라플라시안에서만 경계 유령점의 2배를 빠뜨림")
```

작동 확인 library_test.py — 19.4절: 공개 목록(NodePy)과 포른베르그 스텐실로 넓힌 목록, 유한 범위 전수 확인

```
"""19.4절: 공개 목록(NodePy)으로 넓힌 방법 목록, 포른베르그 스텐실 목록, 유한 범위 전수 확인."""
import sys, warnings, numpy as np
warnings.filterwarnings("ignore")
sys.path.insert(0, "../numide")
from verify import check_claim, library
from schemes_lib import S, ab3_solver
from claim_tools import identify_stencil, exhaustive, fornberg
import space_check as SC
from operators import fdm_vertex
import nodepy.runge_kutta_method as RKm, nodepy.linear_multistep_method as LMm
base, ext = library(False), library(True)
def kept(t, Y, lib):
    return [nm for nm, cl in lib.items() if check_claim(S, t, Y, cl, 1e-12)[0] == "consistent"]
def rk_run(A, b, N=200, bug=None):
    """대상 코드: 아무 명시적 룽게–쿠타 표로 고정 걸음 적분(독립 구현)"""
    A = np.array(A, float); b = np.array(b, float); s = len(b); Y = [S.u0.copy()]; h = S.T / N
    if bug: b = b.copy(); b[bug[0]] += bug[1]
    for _ in range(N):
        y = Y[-1]; K = []
        for i in range(s): K.append(S.F(y + h * sum(A[i, j] * K[j] for j in range(i))))
        Y.append(y + h * sum(b[i] * K[i] for i in range(s)))
    return np.linspace(0, S.T, N + 1), np.array(Y)
print("== 시간: 옛 목록(17개)에 없던 방법을 공개 목록(NodePy, 103개)으로 가려낸다")
for nm in ("Tsit5", "CK5", "BS5", "SSP104", "Merson43", "Lambert65"):
    m = RKm.loadRKM(nm); t, Y = rk_run(np.array(m.A, float), np.array(m.b, float))
```

```
    k0, k1 = kept(t, Y, base), kept(t, Y, ext)
    print(f"  {nm}(200걸음): 옛 목록 {k0 or '없음'} → 넓힌 목록 {k1}")
m = RKm.loadRKM("Tsit5"); t, Y = rk_run(np.array(m.A, float), np.array(m.b, float), bug=(3, 1e-6))
print(f"  Tsit5에서 b_4에 1e-6 오타: 넓힌 목록에서 남는 방법 {kept(t, Y, ext) or '없음(반례: 목록의 어느 방법도 아니다)'}")
def lmm_run(al, be, N=400):
    """대상 코드: 고정 걸음 다단계(시작값은 작은 걸음 RK4로; 암시적이면 뉴턴으로 잔차 1e-14까지)"""
    al = np.array(al, float); be = np.array(be, float); k = len(al) - 1; h = S.T / N; Y = [S.u0.copy()]
    for _ in range(k - 1):
        y = Y[-1]; hh = h / 50
        for _ in range(50):
            k1 = S.F(y); k2 = S.F(y + hh / 2 * k1); k3 = S.F(y + hh / 2 * k2); k4 = S.F(y + hh * k3); y = y + hh / 6 * (k1 + 2 * k2 + 2 * k3 + k4)
        Y.append(y)
    d = len(S.u0)
    for n in range(N - k + 1):
        win = Y[-k:]; rhs = -sum(al[j] * win[j] for j in range(k)) + h * sum(be[j] * S.F(win[j]) for j in range(k))
        z = win[-1].copy()
        for _ in range(30 if be[k] != 0 else 1):
            G = al[k] * z - h * be[k] * S.F(z) - rhs
            if be[k] == 0: z = rhs / al[k]; break
            J = S.Jnum(z.reshape(S.m, S.N)); dz = np.linalg.solve(al[k] * np.eye(d) - h * be[k] * J, G); z = z - dz
            if np.max(np.abs(dz)) < 1e-15: break
        Y.append(z)
    return np.linspace(0, S.T, N + 1), np.array(Y)
for fn, f, k in (("애덤스–배시퍼스", LMm.Adams_Bashforth, 6), ("애덤스–몰턴", LMm.Adams_Moulton, 3), ("뉘스트룀", LMm.Nystrom, 3), ("BDF", LMm.backward_difference_formula, 6)):
    m = f(k); t, Y = lmm_run([float(v) for v in m.alpha], [float(v) for v in m.beta])
    k0, k1 = kept(t, Y, base), kept(t, Y, ext)
    print(f"  {fn} {k}(400걸음): 옛 목록 {k0 or '없음'} → 넓힌 목록 {k1}")
from scipy.integrate import solve_ivp
r = solve_ivp(lambda t_, z: S.F(z), (0, S.T), S.u0, method="DOP853", rtol=1e-8, atol=1e-10)
print(f"  scipy DOP853(적응형): 넓힌 목록에서 남는 방법 {kept(r.t, r.y.T, ext) or '없음(목록 밖: 정확도 판정만 한다)'}")

print("\n== 공간: 포른베르그 스텐실 목록으로 내부 행을 가려낸다(균일 격자)")
x = np.linspace(0, 1, 21); h = x[1] - x[0]
def fd_high(order, bug_row=None):
    """대상 코드: 내부는 중심 (order+1)점, 경계 근처는 3점인 2계 차분"""
    w = [float(v) for v in fornberg(list(range(-(order // 2), order // 2 + 1)), 2)]; L = fdm_vertex(21, 1.0)[0].copy(); r = order // 2
    for i in range(r, 21 - r):
        L[i] = 0; L[i, i - r:i + r + 1] = np.array(w) / h ** 2
    if bug_row is not None: L[bug_row, bug_row + 1] *= 1.0001
    return L
for order, bug in ((2, None), (4, None), (6, None), (4, 9)):
    found, un = identify_stencil(fd_high(order, bug), x)
    kinds = sorted(set(found.values()))
    print(f"  {order}차 내부 스텐실" + (f"(행 {bug}에 1e-4 오타)" if bug is not None else "") + f": 가려낸 스텐실 {kinds}, 내부 행 {len(found)}개 일치" + (f", 맞는 스텐실이 없는 행 {un}(반례)" if un else ""))

print("\n== 유한 범위 전수 확인: 그 범위 안에서는 증명")
def fdm_latent(N):
    L, w, xx = fdm_vertex(N, 1.0)
    if N > 40: L = L.copy(); L[N // 2, N // 2 + 1] *= 1.001
    return L, xx
bad = exhaustive(range(3, 201), lambda N: not SC.check(*fdm_latent(N), "FDM")["ok"])
print(f"  FDM(크기에 따라 숨은 버그), N = 3..200 전부 확인: 반례 {len(bad)}개(N = {bad[0]}..{bad[-1]}), N ≤ 40은 전부 통과(증명)")
def rk4_latent_run(Nst):
    Y = [S.u0.copy()]; hh = S.T / Nst; F = S.F
    for k in range(Nst):
        y = Y[-1]; k1 = F(y); k2 = F(y + hh / 2 * k1); k3 = F(y + hh / 2 * k2)
        if Nst % 7 == 0 and k == Nst - 1: k3 = k2
        k4 = F(y + hh * k3); Y.append(y + hh / 6 * (k1 + 2 * k2 + 2 * k3 + k4))
    return np.linspace(0, S.T, Nst + 1), np.array(Y)
cl = library()["RK4"]
bad = exhaustive(range(20, 121), lambda n: check_claim(S, *rk4_latent_run(n), cl)[0] == "refuted")
print(f"  RK4(7의 배수 걸음에서 숨은 버그), 걸음 수 20..120 전부 확인: 반례가 나는 걸음 수 {bad}; 7의 배수가 아닌 걸음 수는 전부 통과(증명)")
```

build_pdf.py — 이 문서를 만드는 스크립트

```
"""Build the document PDF from content.txt with Python only (reportlab + matplotlib mathtext).

    python3 build_pdf.py [content.txt] [out.pdf]     (defaults: content.txt -> euler_ladder.pdf)

content.txt markup
  ::title T        title line            ::toc            table of contents
  #* T             unnumbered section    # T / ## T       section / subsection (in the contents)
  ### T            run-in heading        - text           bullet ('- □ text' = checklist item)
  $$ expr $$       numbered display equation (matplotlib mathtext syntax)
  $expr$           inline math           **text**         bold
  ::center text    centred line          ::pagebreak      new page
  ::table w1,w2,.. rows 'a | b | c' (first row = header) ... ::end
  ::code path title                      code listing with line numbers
"""
import hashlib, os, re, unicodedata
import numpy as np
from PIL import Image
import matplotlib
matplotlib.rcParams["mathtext.fontset"] = "cm"
from matplotlib.mathtext import MathTextParser
from matplotlib.font_manager import FontProperties
from reportlab.lib import colors
from reportlab.lib.enums import TA_CENTER
from reportlab.lib.pagesizes import A4
from reportlab.lib.styles import ParagraphStyle
from reportlab.lib.units import mm
from reportlab.pdfbase import pdfmetrics
from reportlab.pdfbase.ttfonts import TTFont as RLTTFont
from reportlab.lib.fonts import addMapping
from reportlab.platypus import (BaseDocTemplate, Frame, PageTemplate, Paragraph, Spacer, Table, TableSty
le,
                                PageBreak, Preformatted, KeepTogether, Image as RLImage)
from reportlab.platypus.tableofcontents import TableOfContents

HERE = os.path.dirname(os.path.abspath(__file__))
OUT = os.path.join(HERE, "euler_ladder.pdf")
CACHE = os.path.join(HERE, "_cache"); os.makedirs(CACHE, exist_ok=True)
SERIF = "/usr/share/fonts/opentype/noto/NotoSerifCJK-%s.ttc"
MONO = "/usr/share/fonts/opentype/noto/NotoSansCJK-Regular.ttc"    # face 6 = Noto Sans Mono CJK KR
BODY_PT, CODE_PT = 10.0, 6.6


# ---------------------------------------------------------------------------- fonts
def cff_to_ttf(src, index, chars, dst):
    """Subset one face of a CFF-based CJK collection and convert its outlines to TrueType (reportlab nee
ds glyf)."""
    from fontTools.ttLib import TTFont, newTable
    from fontTools import subset
    from fontTools.pens.cu2quPen import Cu2QuPen
    from fontTools.pens.ttGlyphPen import TTGlyphPen
    f = TTFont(src, fontNumber=index)
    opt = subset.Options(); opt.layout_features = []; opt.name_IDs = ["*"]; opt.notdef_outline = True
    sub = subset.Subsetter(opt); sub.populate(text=chars); sub.subset(f)
    gs = f.getGlyphSet(); order = f.getGlyphOrder(); glyphs = {}
    for name in order:
        pen = TTGlyphPen(gs)
        gs[name].draw(Cu2QuPen(pen, 1.0, reverse_direction=True))
        glyphs[name] = pen.glyph()
    f["loca"] = newTable("loca"); glyf = f["glyf"] = newTable("glyf")
    glyf.glyphOrder = order; glyf.glyphs = glyphs
    del f["CFF "]
    for t in ("VORG", "VVAR", "vhea", "vmtx"):
        if t in f:
            del f[t]
    maxp = f["maxp"] = newTable("maxp"); maxp.tableVersion = 0x00010000
    for k in ("maxZones", "maxTwilightPoints", "maxStorage", "maxFunctionDefs", "maxInstructionDefs",
              "maxStackElements", "maxSizeOfInstructions", "maxComponentElements"):
        setattr(maxp, k, 1 if k == "maxZones" else 0)
    post = f["post"]; post.formatType = 2.0; post.extraNames = []; post.mapping = {}; post.glyphOrder =
order
    f["head"].glyphDataFormat = 0
    f.sfntVersion = "\000\001\000\000"
    glyf.compile(f)
    for name, gl in glyf.glyphs.items():                  # left side bearings from the new outlines
        if hasattr(gl, "xMin"):
            f["hmtx"][name] = (f["hmtx"][name][0], gl.xMin)
    f.save(dst)

```

```
76
77  def setup_fonts(text):
78      chars = "".join(sorted(set(text) | set(map(chr, range(32, 127))) | set("•□")))   # bullet glyphs too
79      key = hashlib.md5(chars.encode()).hexdigest()[:10]
80      for style in ("Regular", "Bold"):
81          dst = os.path.join(CACHE, f"serif-{style}-{key}.ttf")
82          if not os.path.exists(dst):
83              cff_to_ttf(SERIF % style, 1, chars, dst)            # face 1 = Noto Serif CJK KR
84          pdfmetrics.registerFont(RLTTFont("Serif" + ("-B" if style == "Bold" else ""), dst))
85      dst = os.path.join(CACHE, f"mono-{key}.ttf")
86      if not os.path.exists(dst):
87          cff_to_ttf(MONO, 6, chars, dst)
88      pdfmetrics.registerFont(RLTTFont("Mono", dst))
89      global MONO_CMAP
90      from fontTools.ttLib import TTFont as _FT
91      MONO_CMAP = set(_FT(dst).getBestCmap().keys())
92      pdfmetrics.registerFont(RLTTFont("MonoFB", "/usr/share/fonts/truetype/dejavu/DejaVuSansMono.ttf"))
93      for b in (0, 1):
94          for i in (0, 1):
95              addMapping("Serif", b, i, "Serif-B" if b else "Serif")
96
97
98  # ---------------------------------------------------------------------------- math
99  _parser = MathTextParser("agg")
100 DPI = 600
101
102
103 def math_img(expr, size):
104     """Render $expr$ to a cached PNG; return (path, width_pt, height_pt, depth_pt)."""
105     key = hashlib.md5(f"{expr}|{size}".encode()).hexdigest()
106     path = os.path.join(CACHE, key + ".png")
107     r = _parser.parse(f"${expr}$", dpi=DPI, prop=FontProperties(size=size))
108     img = np.asarray(r.image)
109     if not os.path.exists(path):
110         rgba = np.zeros(img.shape + (4,), np.uint8); rgba[..., 3] = img
111         Image.fromarray(rgba, "RGBA").save(path)
112     s = 72.0 / DPI
113     return path, img.shape[1] * s, img.shape[0] * s, r.depth * s
114
115
116 def esc(t):
117     return t.replace("&", "&amp;").replace("<", "&lt;").replace(">", "&gt;")
118
119
120 def rich(text, size=BODY_PT):
121     """Plain text with $math$ and **bold** -> reportlab paragraph markup."""
122     out, bold = [], False
123     for i, part in enumerate(re.split(r"(?<!\\)\$", text)):
124         if i % 2:
125             p, w, h, d = math_img(re.sub(r"\\sum(?=[_^])", r"\\Sigma", part), size)   # inline sums: lim
    its at the side
126             out.append(f'<img src="{p}" width="{w:.2f}" height="{h:.2f}" valign="{-d:.2f}"/>')
127             continue
128         part = re.sub(r"(?<!\*)\*([^*\s][^*]*?)\*(?!\*)", lambda m: "\x01" + m.group(1) + "\x02", esc(pa
    rt))   # *x* -> italic
129         part = re.sub(r"`([^`]+)`", lambda m: '<font face="Mono" color="#1c5cab">' + m.group(1) + "</fon
    t>", part)   # `code`
130         pieces = part.split("**")
131         for j, piece in enumerate(pieces):
132             if j:
133                 bold = not bold
134                 out.append("<b>" if bold else "</b>")
135             out.append(piece.replace("\x01", "<i>").replace("\x02", "</i>"))
136     if bold:
137         out.append("</b>")
138     return "".join(out)
139
140
141 # ---------------------------------------------------------------------------- styles
142 ST = dict(
143     body=ParagraphStyle("body", autoLeading="max", fontName="Serif", fontSize=BODY_PT, leading=BODY_PT *
     1.75, spaceAfter=3,
144                         allowWidows=0),
145     bullet=ParagraphStyle("bullet", autoLeading="max", fontName="Serif", fontSize=BODY_PT, leading=BODY_
    PT * 1.75, leftIndent=14,
146                           bulletIndent=4, bulletFontName="Serif", spaceAfter=2),
147     cell=ParagraphStyle("cell", autoLeading="max", fontName="Serif", fontSize=8.6, leading=12.5),
148     cellh=ParagraphStyle("cellh", autoLeading="max", fontName="Serif-B", fontSize=8.6, leading=12.5),
149     h0=ParagraphStyle("h0", fontName="Serif-B", fontSize=17, leading=26, alignment=TA_CENTER),
150     h1=ParagraphStyle("h1", keepWithNext=1, fontName="Serif-B", fontSize=14, leading=20, spaceBefore=14,
```

```
     spaceAfter=8),
    h2=ParagraphStyle("h2", keepWithNext=1, fontName="Serif-B", fontSize=11.5, leading=17, spaceBefore=1
0, spaceAfter=5),
    h3=ParagraphStyle("h3", keepWithNext=1, fontName="Serif-B", fontSize=10.5, leading=16, spaceBefore=8
, spaceAfter=3),
    center=ParagraphStyle("center", fontName="Serif", fontSize=BODY_PT, leading=BODY_PT * 1.8, alignment
=TA_CENTER),
    eqn=ParagraphStyle("eqn", fontName="Serif", fontSize=BODY_PT, leading=BODY_PT * 1.4, alignment=2),
    code=ParagraphStyle("code", fontName="Mono", fontSize=CODE_PT, leading=CODE_PT * 1.3),
    toc0=ParagraphStyle("toc0", fontName="Serif-B", fontSize=10, leading=15),
    toc1=ParagraphStyle("toc1", fontName="Serif", fontSize=9, leading=12.5, leftIndent=16),
)


class Doc(BaseDocTemplate):
    def __init__(self, path, title=""):
        super().__init__(path, pagesize=A4, leftMargin=25 * mm, rightMargin=25 * mm, topMargin=24 * mm,
                         bottomMargin=24 * mm, title=title)
        fr = Frame(self.leftMargin, self.bottomMargin, self.width, self.height, id="f")
        self.addPageTemplates([PageTemplate("p", [fr], onPage=self.footer)])

    @staticmethod
    def footer(c, doc):
        c.setFont("Serif", 9); c.drawCentredString(A4[0] / 2, 12 * mm, str(doc.page))

    def afterFlowable(self, f):
        lvl = getattr(f, "_toc", None)
        if lvl is not None:
            key = "h%d" % id(f)
            self.canv.bookmarkPage(key)
            if lvl <= TOC_MAX[0]:
                self.notify("TOCEntry", (lvl, f._toc_text, self.page, key))
            self.canv.addOutlineEntry(f._toc_text, key, level=lvl, closed=lvl >= 1 and SHIFT[0] == 1)


def heading(text, style, lvl):
    p = Paragraph(rich(text, style.fontSize), style)
    plain = text
    for a, b in (("^\\circ", "°"), ("\\pi", "π"), ("\\theta", "θ"), ("$", "")):
        plain = plain.replace(a, b)
    p._toc, p._toc_text = lvl, plain
    return p


EQN = [0]
SHIFT = [0]            # 1 when the text has part headings (#! ...): # -> level 1, ## -> level 2
TOC_MAX = [1]
FRAME_W = [0.0]


def equation(expr):
    EQN[0] += 1
    p, w, h, d = math_img(re.sub(r"\\frac(?=\{)", r"\\dfrac", expr), BODY_PT + 1.5)
    room = FRAME_W[0] - 2 * 40 - 12
    if w > room:                                          # shrink an over-wide equation to fit
        w, h, d = w * room / w, h * room / w, d * room / w
    img = RLImage(p, width=w, height=h)
    t = Table([["", img, Paragraph(f"({EQN[0]})", ST["eqn"])]], colWidths=[40, FRAME_W[0] - 80, 40])
    t.setStyle(TableStyle([("VALIGN", (0, 0), (-1, -1), "MIDDLE"), ("ALIGN", (1, 0), (1, 0), "CENTER"),
                           ("TOPPADDING", (0, 0), (-1, -1), 3),
                           ("BOTTOMPADDING", (0, 0), (-1, -1), 3)]))
    return t


def split_cells(line):
    cells, cur, inm = [], "", False
    for ch in line:
        if ch == "$":
            inm = not inm
        if ch == "|" and not inm:
            cells.append(cur.strip()); cur = ""
        else:
            cur += ch
    cells.append(cur.strip())
    return cells


def table(widths, rows, frame_w):
    ws = [float(x) for x in widths.split(",")]
    data = []
```

```
    for r, line in enumerate(rows):
        cells = split_cells(line); cells += [""] * (len(ws) - len(cells))
        st = ST["cellh"] if r == 0 else ST["cell"]
        data.append([Paragraph(rich(c, 8.6), st) for c in cells[:len(ws)]])
    t = Table(data, colWidths=[w * frame_w for w in ws], repeatRows=1, hAlign="CENTER")
    t.setStyle(TableStyle([("LINEABOVE", (0, 0), (-1, 0), 0.9, colors.black),
                           ("LINEBELOW", (0, 0), (-1, 0), 0.5, colors.black),
                           ("LINEBELOW", (0, -1), (-1, -1), 0.9, colors.black),
                           ("VALIGN", (0, 0), (-1, -1), "MIDDLE"),
                           ("TOPPADDING", (0, 0), (-1, -1), 2.5), ("BOTTOMPADDING", (0, 0), (-1, -1), 2.
5)]))
    return [Spacer(1, 4), t, Spacer(1, 6)]


def mono_flow(lines):
    """고정폭 블록: CJK 고정폭 글꼴에 없는 글자(ℝ, ₀, ⁻ 등)는 DejaVu Sans Mono로 찍는다."""
    cm = globals().get("MONO_CMAP", set())
    if any(ord(ch) not in cm for l_ in lines for ch in l_ if ch not in " \n"):
        from reportlab.platypus import XPreformatted
        from xml.sax.saxutils import escape as _esc
        return XPreformatted("\n".join("".join(_esc(ch) if (ord(ch) in cm or ch == " ") else '<font name
="MonoFB">' + _esc(ch) + "</font>" for ch in l_) for l_ in lines), ST["code"])
    return Preformatted("\n".join(lines), ST["code"])


def code_block(path, title):
    src = open(os.path.join(HERE, path)).read().rstrip("\n").split("\n")
    lines, width = [], 104
    for i, l in enumerate(src, 1):
        l = l.replace("\t", "    ")
        first = True
        while True:                                  # wrap by display width (East Asian wide character
s count 2)
            cut, wsum = 0, 0
            while cut < len(l) and wsum + (2 if unicodedata.east_asian_width(l[cut]) in "WF" else 1) <=
width:
                wsum += 2 if unicodedata.east_asian_width(l[cut]) in "WF" else 1; cut += 1
            lines.append(("%4d  " % i if first else "      ") + l[:cut])
            l = l[cut:]; first = False
            if not l:
                break
    from reportlab.platypus import KeepTogether
    from reportlab.lib.styles import ParagraphStyle as _PS
    h3n = _PS("h3n", parent=ST["h3"], keepWithNext=0)
    cm = globals().get("MONO_CMAP", set())
    if any(ord(ch) not in cm for l_ in lines for ch in l_ if ch not in " \n"):
        from reportlab.platypus import XPreformatted
        from xml.sax.saxutils import escape as _esc
        def mk(ls):     # glyphs missing in the CJK mono face (e.g. ℝ, ₀) are set in DejaVu Sans Mono
            return XPreformatted("\n".join("".join(_esc(ch) if (ord(ch) in cm or ch == " ") else '<font
name="MonoFB">' + _esc(ch) + "</font>" for ch in l_) for l_ in ls), ST["code"])
    else:
        mk = lambda ls: Preformatted("\n".join(ls), ST["code"])
    head = KeepTogether([Paragraph(rich("**" + title + "**"), h3n), mk(lines[:6])])
    rest = [mk(lines[6:])] if len(lines) > 6 else []
    return [head] + rest + [Spacer(1, 8)]


def build(content="content.txt", out=OUT):
    text = open(os.path.join(HERE, content)).read()
    first = re.search(r"^::title (.*)$", text, re.M)
    codes = re.findall(r"^::code (\S+)", text, re.M)
    setup_fonts(text + "".join(open(os.path.join(HERE, c)).read() for c in codes))
    doc = Doc(out, re.sub(r"\$|\\", "", first.group(1)) if first else ""); W = doc.width; FRAME_W[0] = W
    story, lines, i = [], text.split("\n"), 0
    if any(l.startswith("#! ") for l in lines): SHIFT[0] = 1
    while i < len(lines):
        ln = lines[i].rstrip(); i += 1
        if not ln:
            continue
        if ln.startswith("::title "):
            story.append(Paragraph(rich(ln[8:], 17), ST["h0"]))
        elif ln == "::toc":
            toc = TableOfContents(); toc.levelStyles = [ST["toc0"], ST["toc1"], ST["toc1"]]
            story += [Spacer(1, 10), Paragraph("차례", ParagraphStyle("tochd", parent=ST["h1"], keepWith
Next=0)), toc]
        elif ln == "::pagebreak":
            from reportlab.platypus import CondPageBreak
            story.append(CondPageBreak(doc.height * 0.3))
        elif ln.startswith("::center "):
```

```
            story.append(Paragraph(rich(ln[9:]), ST["center"]))
        elif ln.startswith("::table "):
            rows = []
            while lines[i].strip() != "::end":
                rows.append(lines[i]); i += 1
            i += 1
            story += table(ln[8:], rows, W)
        elif ln.startswith("::image "):
            _, path, frac, cap = ln.split(" ", 3)
            from PIL import Image as _PI
            iw, ih = _PI.open(os.path.join(HERE, path)).size; wd = W * float(frac)
            story += [Spacer(1, 4), RLImage(os.path.join(HERE, path), width=wd, height=wd * ih / iw),
                      Paragraph(rich(cap), ST["center"]), Spacer(1, 6)]
        elif ln == "::pre":                             # preformatted block (no line numbers) up to ::end
            blk = []
            while lines[i].rstrip() != "::end":
                l = lines[i].rstrip("\n"); i += 1
                while True:
                    cut, wsum = 0, 0
                    while cut < len(l) and wsum + (2 if unicodedata.east_asian_width(l[cut]) in "WF" else 1) <= 104:
                        wsum += 2 if unicodedata.east_asian_width(l[cut]) in "WF" else 1; cut += 1
                    blk.append(l[:cut]); l = "      " + l[cut:] if l[cut:] else ""
                    if not l: break
            i += 1
            story += [Spacer(1, 3), mono_flow(blk), Spacer(1, 6)]
        elif ln.startswith("::code "):
            _, path, title = ln.split(" ", 2)
            story += code_block(path, title)
        elif ln.startswith("#! "):
            story.append(heading(ln[3:], ST["h0"], 0))
        elif ln.startswith("#* "):
            story.append(heading(ln[3:], ST["h1"], SHIFT[0]))
        elif ln.startswith("### "):
            story.append(Paragraph(rich(ln[4:], 10.5), ST["h3"]))
        elif ln.startswith("## "):
            story.append(heading(ln[3:], ST["h2"], 1 + SHIFT[0]))
        elif ln.startswith("# "):
            story.append(heading(ln[2:], ST["h1"], SHIFT[0]))
        elif ln.startswith("$$") and ln.endswith("$$"):
            story.append(equation(ln[2:-2].strip()))
        elif ln.startswith("- "):
            item, mark = ln[2:], "•"
            if item.startswith("□ "):                        # checklist item: the box is the bullet
                item, mark = item[2:], "□"
            story.append(Paragraph(rich(item), ST["bullet"], bulletText=mark))
        else:
            story.append(Paragraph(rich(ln), ST["body"]))
    from reportlab.platypus import Spacer as _Sp, CondPageBreak as _CPB
    while story and isinstance(story[-1], (_Sp, _CPB, PageBreak)): story.pop()      # no blank last page
    doc.multiBuild(story)
    try:                                    # a split at the very bottom of the last page can leave an empty page: drop it
        import pypdf
        r = pypdf.PdfReader(out)
        if len(r.pages) > 1 and r.pages[-1].extract_text().strip().isdigit():
            w = pypdf.PdfWriter(clone_from=r); w.remove_page(len(r.pages) - 1)
            with open(out, "wb") as fh: w.write(fh)
    except ImportError:
        pass
    print("wrote", out, "equations:", EQN[0])


if __name__ == "__main__":
    import sys
    build(*sys.argv[1:3])
```